

Most importantly, a subclass extends a base class in either or both of the following ways (which will be explained soon):

- A subclass can *redefine* a method that's defined in the base class. That is, a subclass can provide a method with same name as one in the base class but with different functionality. This is called *overriding* a method. When client code makes a call to an overridden method, the method in the subclass is called. (However, the code of the method in the subclass can still call the method of the same name in the base class.)
- A subclass can add new methods and instance variables that do not appear in the base class.

One way to think of a subclass is with the phrase *coding by difference*. Since the subclass inherits all the instance variables and methods of the base class, it does not need to repeat all of that code; the subclass only needs to contain code that differentiates it from the base class. The code of the subclass therefore only contains new instance variables (and their initialization), overriding methods, and/or new methods not found in the base class.

Implementing Inheritance

The syntax of inheritance in Python is simple and elegant. The base class does not need to *know* that it is being used as a base class. Only the subclass needs to indicate that it wants to inherit from a base class. Here is the general syntax:

```
class <BaseClassName>():  
    # BaseClass methods  
  
class <SubClassName>(<BaseClassName>):  
    # SubClass methods
```

In the `class` statement of the subclass, within the parentheses you specify the name of the base class it should inherit from. In this case, we want the subclass `<SubClassName>` to inherit from the base class `<BaseClassName>`.